PROGRAM:

```
%
================================================================
=======
% Combined Decimation and Interpolation (Rational Rate Conversion)
% Sampling Rate Conversion Factor: L/M = 2/3
%
================================================================
=======
clear; clc; close all;
%% Given Data
Fs_in = 18000;         % Input Sampling Rate = 18 kHz
L     = 2;             % Interpolation Factor
M     = 3;             % Decimation Factor
%% Task 5: Determine the Final Sampling Rate
% Intermediate rate after upsampling: Fs_inter = L * Fs_in = 36 kHz
Fs_inter = L * Fs_in;
% Final sampling rate after downsampling: Fs_out = Fs_inter / M = 12 kHz
Fs_out   = Fs_in * (L / M);
fprintf('=== SAMPLING RATE ANALYSIS ===\n');
fprintf('Input Sampling Rate  (Fs_in)  : %8.2f Hz\n', Fs_in);
fprintf('Interpolation Factor (L)      : %8d\n', L);
fprintf('Decimation Factor    (M)      : %8d\n', M);
fprintf('Intermediate Rate    (Fs_inter): %8.2f Hz\n', Fs_inter);
fprintf('Final Sampling Rate  (Fs_out) : %8.2f Hz\n\n', Fs_out);
%% Input Signal Generation
% Generate a multi-tone signal containing 800 Hz and 2.5 kHz
% Note: Output Nyquist limit is Fs_out / 2 = 6 kHz. Both tones fit safely.
```

```
t_duration = 0.01; % 10 ms duration for time-domain viewing
t_in = 0 : 1/Fs_in : t_duration - 1/Fs_in;
f1 = 800;   % Tone 1: 800 Hz
f2 = 2500;  % Tone 2: 2500 Hz
x_in = sin(2*pi*f1*t_in) + 0.6*cos(2*pi*f2*t_in);
%% Task 1: Design Anti-Alias / Anti-Imaging Lowpass Filter
% Cutoff frequency must remove images created by upsampling (pi/L)
% and prevent aliasing from downsampling (pi/M).
% Cutoff in normalized units (0 to 1, where 1 is Fs_inter/2):
fc_norm = 1 / max(L, M);  % fc = 1/3 of Fs_inter/2 = 6 kHz
N = 60;                   % FIR Filter order (even for symmetric Type I)
% Linear phase FIR filter designed with Hamming window
% Scale coefficients by L to compensate for zero insertion amplitude drop
b = L * fir1(N, fc_norm);
%% Task 3: Implement Interpolation (Upsampling + Lowpass Filtering)
% Step 3A: Zero Insertion (Upsample by L)
x_up = upsample(x_in, L);
% Step 3B: Anti-Imaging Filtering
x_interp = filter(b, 1, x_up);
%% Task 2: Perform Decimation (Downsampling by M)
% Keep 1 out of every M samples
x_out = downsample(x_interp, M);
% Define time vector for output signal
t_out = 0 : 1/Fs_out : (length(x_out)-1)/Fs_out;
% Account for filter group delay (N/2 samples at Fs_inter)
group_delay_seconds = (N / 2) / Fs_inter;
%% Task 4 & 6: Frequency Spectrum Analysis and Comparison
```

```
% FFT parameters
NFFT = 2048;
% Input Spectrum
X_in = fftshift(fft(x_in, NFFT));
f_in_axis = linspace(-Fs_in/2, Fs_in/2 - Fs_in/NFFT, NFFT);
% Intermediate Spectrum (Filtered Upsampled Signal)
X_interp = fftshift(fft(x_interp, NFFT));
f_interp_axis = linspace(-Fs_inter/2, Fs_inter/2 - Fs_inter/NFFT, NFFT);
% Output Spectrum
X_out = fftshift(fft(x_out, NFFT));
f_out_axis = linspace(-Fs_out/2, Fs_out/2 - Fs_out/NFFT, NFFT);
%% Plotting Results
% 1. Time-Domain Signal Comparison
figure('Name', 'Multirate System - Time Domain', 'NumberTitle', 'off');
subplot(2,1,1);
stem(t_in*1e3, x_in, 'filled', 'b', 'LineWidth', 1.2);
hold on;
plot(t_in*1e3, x_in, 'b--', 'LineWidth', 0.8);
grid on;
title('Input Signal x[n] (F_{s,in} = 18 kHz)');
xlabel('Time (ms)');
ylabel('Amplitude');
legend('Samples', 'Envelope');
subplot(2,1,2);
% Align time to compensate for FIR filter delay in display
stem((t_out - group_delay_seconds)*1e3, x_out, 'filled', 'r', 'LineWidth', 1.2);
hold on;
```

```
plot((t_out - group_delay_seconds)*1e3, x_out, 'r--', 'LineWidth', 0.8);
grid on;
title('Resampled Output Signal y[n] (F_{s,out} = 12 kHz)');
xlabel('Time (ms, Delay Compensated)');
ylabel('Amplitude');
legend('Samples', 'Envelope');
% 2. Spectrum Analysis Comparison
figure('Name', 'Multirate System - Frequency Spectra', 'NumberTitle', 'off');
subplot(3,1,1);
plot(f_in_axis/1e3, abs(X_in)/max(abs(X_in)), 'b', 'LineWidth', 1.2);
grid on;
title('Input Spectrum |X_{in}(f)| (F_{s,in} = 18 kHz)');
xlabel('Frequency (kHz)');
ylabel('Normalized Magnitude');
xlim([-Fs_in/2e3, Fs_in/2e3]);
subplot(3,1,2);
plot(f_interp_axis/1e3, abs(X_interp)/max(abs(X_interp)), 'g', 'LineWidth', 1.2);
grid on;
title(['Interpolated & Filtered Spectrum (F_{s,inter} = ' num2str(Fs_inter/1e3) ' kHz)']);
xlabel('Frequency (kHz)');
ylabel('Normalized Magnitude');
xlim([-Fs_inter/2e3, Fs_inter/2e3]);
subplot(3,1,3);
plot(f_out_axis/1e3, abs(X_out)/max(abs(X_out)), 'r', 'LineWidth', 1.2);
grid on;
figure('Name', 'Anti-Alias / Anti-Imaging Filter Response', 'NumberTitle', 'off');
xline((Fs_inter/2 * fc_norm)/1e3, 'r--', 'Cutoff Frequency (6 kHz)');
```

xline((Fs_inter/2 * fc_norm)/1e3, 'r--', 'Cutoff Frequency (6 kHz)');

TOOL USED: Online Matlab simulator

OUTPUT: